

Ecole

백준 문제 풀이

2025.09.06

Python

수 정렬하기

**문제[000] 수 정렬하기**

시간 제한 2초 | 백준 온라인 저지 2750번

N개의 수가 주어졌을 때 이를 오름차순 정렬하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 수의 개수 N ($1 \leq N \leq 1,000,000$), 2번째 줄부터는 N개의 줄에 숫자가 주어진다. 이 수는 절댓값이 1,000,000보다 작거나 같은 정수다. 수는 중복되지 않는다.

↑ 출력

1번째 줄부터 N개의 줄에 오름차순 정렬한 결과를 1줄에 1개씩 출력한다.

예제 입력 1

5
5
2
3
4
1

예제 출력 1

1
2
3
4
5



병합 정렬 구현

병합 정렬은 '분할 정복' 전략을 사용하는 대표적인 정렬 알고리즘

큰 문제를 작은 문제로 분할하여
해결 후 그 결과를 다시 합쳐
원래의 큰 문제를 해결하는 방식

```
import sys

def merge(left, right):
    result = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    return merge(merge_sort(left), merge_sort(right))

n = int(input())
numList = []

for i in range(n):
    numList.append(int(input()))

result = merge_sort(numList)

for nums in result:
    print(nums)
```

파이썬 내장 sort() 함수 사용

```
n = int(input())
result = []
for i in range(n):
    result.append(int(input()))
result.sort()

for i in range(n):
    print(result[i])
```

Java, JS

수 정렬하기



Java

```
public class P001 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));

        int n = Integer.parseInt(br.readLine());

        int[] numList = new int[n];
        for (int i = 0; i < n; i++) {
            numList[i] = Integer.parseInt(br.readLine());
        }

        int[] result = mergeSort(numList);

        for (int num : result) {
            bw.write(str num + "\n");
        }

        bw.flush();
        bw.close();
        br.close();
    }

    private static int[] mergeSort(int[] arr) { 3 usages new *
        // 배열의 크기가 1 이하이면 이미 정렬된 것으로 생각
        if (arr.length <= 1) {
            return arr;
        }

        // 1. 분할 (Divide)
        int mid = arr.length / 2;

        int[] left = Arrays.copyOfRange(arr, from: 0, mid);
        int[] right = Arrays.copyOfRange(arr, mid, arr.length);

        // 2. 정복 (Conquer) & 결합 (Combine)
        return merge(mergeSort(left), mergeSort(right));
    }
}
```

```
private static int[] mergeSort(int[] arr) { 3 usages new *
    // 배열의 크기가 1 이하이면 이미 정렬된 것으로 생각
    if (arr.length <= 1) {
        return arr;
    }

    // 1. 분할 (Divide)
    int mid = arr.length / 2;

    int[] left = Arrays.copyOfRange(arr, from: 0, mid);
    int[] right = Arrays.copyOfRange(arr, mid, arr.length);

    // 2. 정복 (Conquer) & 결합 (Combine)
    return merge(mergeSort(left), mergeSort(right));
}

private static int[] merge(int[] left, int[] right) { 1 usage new *
    int[] result = new int[left.length + right.length];
    int i = 0, j = 0, k = 0;

    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            result[k++] = left[i++];
        } else {
            result[k++] = right[j++];
        }
    }

    while (i < left.length) {
        result[k++] = left[i++];
    }

    while (j < right.length) {
        result[k++] = right[j++];
    }

    return result;
}
```

JavaScript

```
const fs = require('fs');
const input :string[] = fs.readFileSync('/dev/stdin').toString().trim().split(separator: '\n');

const n :number = parseInt(input[0], radix: 10);
const numList :number[] = input.slice(1).map(Number);

function merge(left, right) :any[] { Show usages
    const result :any[] = [];
    let i :number = 0;
    let j :number = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            result.push(left[i]);
            i++;
        } else {
            result.push(right[j]);
            j++;
        }
    }

    return result.concat(left.slice(i), right.slice(j));
}

function mergeSort(arr) :any[]... { Show usages
    if (arr.length <= 1) {
        return arr;
    }

    // 1. 분할 (Divide)
    const mid :number = Math.floor(x: arr.length / 2);
    const left = arr.slice(0, mid);
    const right = arr.slice(mid);

    // 2. 정복 (Conquer) & 결합 (Combine)
    return merge(mergeSort(left), mergeSort(right));
}

const sortedList :any[]... = mergeSort(numList);

console.log(sortedList.join('\n'));
```

Python

숫자의 합 구하기



문제[001] 숫자의 합 구하기

시간 제한 1초 | 난이도 브론즈II | 백준 온라인 저지 11720번

N개의 숫자가 공백 없이 써 있다. 이 숫자를 모두 합해 출력하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 숫자의 개수 N ($1 \leq N \leq 100$), 2번째 줄에 숫자 N 개가 공백 없이 주어진다.

↑ 출력

입력으로 주어진 숫자 N 개의 합을 출력한다.

예제 입력 1

```
1 // 숫자의 개수
1 // 공백 없이 주어진 N개의 숫자
```

예제 출력 1

1

예제 입력 2

```
5
54321
```

예제 출력 2

15

예제 입력 3

```
25
700000000000000000000000000000
```

예제 출력 3

7

예제 입력 4

```
11
10987654321
```

예제 출력 4

46



Python

```
n = int(input())
numList = list(input())
result = 0
for i in numList:
    result += int(i)
print(result)
```

Java

```
public class P002 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int n = Integer.parseInt(br.readLine());

        String numString = br.readLine();

        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += numString.charAt(i) - '0';
        }

        System.out.println(sum);

        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');
const input : string[] = fs.readFileSync('/dev/stdin').toString().trim().split( separator: '\n' );

const n : number = parseInt(input[0], radix: 10);
const numString : string = input[1];

const sum : number = numString.split( separator: '' ).reduce((acc : number , current : string ) => {
    return acc + parseInt(current, radix: 10);
}, 0);

console.log(sum);|
```

Python

구간 합 구하기



핵심

문제[003] 구간 합 구하기

시간 제한 0.5초 | 난이도 실버Ⅲ | 백준 온라인 저지 11659번

수 N 개가 주어졌을 때 i 번째 수에서 j 번째 수까지의 합을 구하는 프로그램을 작성하시오.

↓ 입력

1번째 줄에 수의 개수 N ($1 \leq N \leq 100,000$), 합을 구해야 하는 횟수 M ($1 \leq M \leq 100,000$), 2번째 줄에 N 개의 수가 주어진다. 각 수는 1,000보다 작거나 같은 자연수다. 3번째 줄부터는 M 개의 줄에 합을 구해야 하는 구간 i 와 j 가 주어진다.

↑ 출력

총 M 개의 줄에 입력으로 주어진 i 번째 수에서 j 번째 수까지의 합을 출력한다.

예제 입력 1

```
5 3          // 데이터의 개수, 질의 개수
5 4 3 2 1    // 구간 합을 구할 대상 배열
1 3
2 4
5 5
```

예제 출력 1

```
12
9
1
```



합 배열 공식

$$S[i] = S[i-1] + A[i]$$

인덱스	1	2	3	4	5
배열 A	5	4	3	2	1
합 배열 S	5	9	12	14	15

구간 합 공식

$$S[j] - S[i-1]$$

질의1(1, 3): $S[3] - S[0] = 12 - 0 = 12$

질의2(2, 4): $S[4] - S[1] = 14 - 5 = 9$

질의3(5, 5): $S[5] - S[4] = 15 - 14 = 1$

```
import sys
input = sys.stdin.readline
n, m = map(int, input().split())
numList = list(map(int, input().split()))
prev = [0]
result = 0

for i in range(n):
    result += numList[i]
    prev.append(result)

for num in range(m):
    i, j = map(int, input().split())
    print(prev[j] - prev[i-1])
```

Java, JS

구간 합 구하기



Java

```
public class P003 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());

        int n = Integer.parseInt(st.nextToken());
        int m = Integer.parseInt(st.nextToken());

        // 구간 합(Prefix Sum) 배열 생성
        // prefixSum[i]는 1번째부터 i번째 원소까지의 누적 합을 저장
        long[] prefixSum = new long[n + 1];
        st = new StringTokenizer(br.readLine());
        for (int i = 1; i <= n; i++) {
            prefixSum[i] = prefixSum[i - 1] + Integer.parseInt(st.nextToken());
        }

        for (int k = 0; k < m; k++) {
            st = new StringTokenizer(br.readLine());
            int i = Integer.parseInt(st.nextToken());
            int j = Integer.parseInt(st.nextToken());

            // 구간 합 공식: S[j] - S[i-1]
            long result = prefixSum[j] - prefixSum[i - 1];
            bw.write(st + result + "\n");
        }

        bw.flush();
        bw.close();
        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');
const input : string[] = fs.readFileSync('/dev/stdin').toString().trim().split( separator: '\n');

const [n : number , m : number] = input[0].split( separator: ' ').map(Number);
const numbers : number[] = input[1].split( separator: ' ').map(Number);

// 구간 합 배열 생성
const prefixSum : any[] = new Array( arrayLength: n + 1).fill( value: 0);
for (let i : number = 0; i < n; i++) {
    prefixSum[i + 1] = prefixSum[i] + numbers[i];
}

const results : any[] = [];
for (let k : number = 2; k < m + 2; k++) {
    const [i : number , j : number] = input[k].split( separator: ' ').map(Number);
    // 구간 합 공식: S[j] - S[i-1]
    const result : number = prefixSum[j] - prefixSum[i - 1];
    results.push(result);
}

console.log(results.join('\n'));
```

Python

연속된 자연수의 합 구하기

**문제[006] 연속된 자연수의 합 구하기**

시간 제한 2초 | 난이도 실버V | 백준 온라인 저지 2018번

어떠한 자연수 N 은 몇 개의 연속된 자연수의 합으로 나타낼 수 있다. 당신은 어떤 자연수 $N(1 \leq N \leq 10,000,000)$ 을 몇 개의 연속된 자연수의 합으로 나타내는 가짓수를 알고 싶다. 이때 사용하는 자연수는 N 이어야 한다. 예를 들어 15를 나타내는 방법은 15, $7 + 8$, $4 + 5 + 6$, $1 + 2 + 3 + 4 + 5$ 이다. 반면, 10을 나타내는 방법은 10, $1 + 2 + 3 + 4$ 이다. N 을 입력받아 연속된 자연수의 합으로 나타내는 가짓수를 출력하는 프로그램을 작성하시오.

↓ 입력1번째 줄에 정수 N 이 주어진다.**↑ 출력**입력된 자연수 N 을 연속된 자연수의 합으로 나타내는 가짓수를 출력한다.**예제 입력 1**

15 // N

예제 출력 1

4



투 포인터 혹은 슬라이딩 윈도우

확장과 축소 두 가지 동작

확장

- end 포인터를 오른쪽으로 움직이며 합을 누적

축소

- 숫자 합이 n보다 커지면 합이 n보다 작거나 같아질때까지 start 포인터를 이동

```
n = int(input())
cnt, current_sum, start = 0, 0, 1

# end 포인터가 1부터 n까지 순차적으로 이동
for end in range(1, n+1):
    # 윈도우 확장: 현재 end값에 합계를 더함
    current_sum += end

    # 윈도우 축소: 합계가 n보다 크면, n보다 작거나 같아질때까지 start를 이동시키며 값을 감소
    while current_sum > n:
        current_sum -= start
        start += 1

    # 값 확인: 합계가 n경우 cnt += 1
    if current_sum == n:
        cnt += 1

print(cnt)
```

Java, JS

연속된 자연수의 합 구하기



Java

```
public class P005 {  
    public static void main(String[] args) throws IOException {  
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
  
        int n = Integer.parseInt(br.readLine());  
        int[] numbers = new int[n];  
  
        StringTokenizer st = new StringTokenizer(br.readLine());  
        for (int i = 0; i < n; i++) {  
            numbers[i] = Integer.parseInt(st.nextToken());  
        }  
  
        Arrays.sort(numbers);  
  
        int count = 0;  
  
        for (int k = 0; k < n; k++) {  
            int target = numbers[k];  
            int start = 0;  
            int end = n - 1;  
  
            while (start < end) {  
                // Case 1: 포인터가 자기 자신을 가리키는 경우, 건너뛴다.  
                if (start == k) {  
                    start++;  
                    continue;  
                }  
                if (end == k) {  
                    end--;  
                    continue;  
                }  
                int currentSum = numbers[start] + numbers[end];  
                // Case 2: 두 수의 합이 목표값과 같은 경우  
                if (currentSum == target) {  
                    count++;  
                    break;  
                }  
                // Case 3: 합이 목표값보다 작은 경우  
                else if (currentSum < target) {  
                    start++;  
                }  
                // Case 4: 합이 목표값보다 큰 경우  
                else {  
                    end--;  
                }  
            }  
        }  
        System.out.println(count);  
        br.close();  
    }  
}
```

JavaScript

```
const fs = require('fs');  
const input : string[] = fs.readFileSync('/dev/stdin').toString().trim().split(' ');  
  
const n : number = parseInt(input[0], 10);  
const numbers : number[] = input[1].split(' ').map(Number);  
  
// 숫자 오름차순으로 정렬  
numbers.sort((a : number, b : number) => a - b);  
  
let count : number = 0;  
  
for (let k : number = 0; k < n; k++) {  
    const target : number = numbers[k];  
    let start : number = 0;  
    let end : number = n - 1;  
  
    while (start < end) {  
        // 포인터가 자기 자신을 가리키는 경우 스킵  
        if (start === k) {  
            start++;  
            continue;  
        }  
        if (end === k) {  
            end--;  
            continue;  
        }  
        const currentSum : number = numbers[start] + numbers[end];  
  
        if (currentSum === target) {  
            count++;  
            break;  
        } else if (currentSum < target) {  
            start++;  
        } else {  
            end--;  
        }  
    }  
}  
console.log(count);
```

Python

'좋은 수' 구하기



핵심

문제[008] '좋은 수' 구하기

시간 제한 2초 | 난이도 골드 IV | 백준 온라인 저지 1253번

주어진 N개의 수에서 다른 두 수의 합으로 표현되는 수가 있다면 그 수를 '좋은 수'라고 한다. N개의 수 중 좋은 수가 총 몇 개인지 출력하시오.

↓ 입력

1번째 줄에 수의 개수 N ($1 \leq N \leq 2,000$), 2번째 줄에 N개의 수의 값(A_i)이 주어진다($|A_i| \leq 1,000,000,000$, A_i 는 정수).

↑ 출력

좋은 수의 개수를 출력한다.

예제 입력 1

```
10 // 수의 개수
1 2 3 4 5 6 7 8 9 10
```

예제 출력 1

```
8
```



앞선 문제들과 다르게 정렬 후 시작

numList가 [1, 2, 3, 4]로 가정

초기 target = 1(index 0)

start(index 0), end(index 4) -> start, target
의 index 겹치므로 start 오른쪽으로 이동

⇒ start(index 1) + end(index 4) = 7

계산 결과가 target보다 크므로 합을 줄이기
위해 end포인터 index 감소

이를 반복하면 while문에 걸려 종료

for문이 계속 돌며 target과 일치하는
경우 추가

```
n = int(input())
numList = list(map(int, input().split()))
numList.sort()

result = 0

for i in range(n):
    target = numList[i]
    start, end = 0, n - 1

    while start < end:
        # 포인터가 자기 자신을 가리키는 경우 스킵
        if start == i:
            start += 1
            continue
        if end == i:
            end -= 1
            continue

        current_sum = numList[start] + numList[end]

        # 두 수의 합이 목표값과 같은 경우
        if current_sum == target:
            result += 1
            break
        # 합이 목표값보다 작은 경우
        elif current_sum < target:
            start += 1
        # 합이 목표값보다 큰 경우
        else:
            end -= 1

print(result)
```

Java, JS

'좋은 수' 구하기



Java

```
public class P005 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int n = Integer.parseInt(br.readLine());
        int[] numbers = new int[n];

        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) {
            numbers[i] = Integer.parseInt(st.nextToken());
        }

        Arrays.sort(numbers);

        int count = 0;

        for (int k = 0; k < n; k++) {
            int target = numbers[k];
            int start = 0;
            int end = n - 1;

            while (start < end) {
                // Case 1: 포인터가 자기 자신을 가리키는 경우, 건너뛴다.
                if (start == k) {
                    start++;
                    continue;
                }
                if (end == k) {
                    end--;
                    continue;
                }
                int currentSum = numbers[start] + numbers[end];
                // Case 2: 두 수의 합이 목표값과 같은 경우
                if (currentSum == target) {
                    count++;
                    break;
                }
                // Case 3: 합이 목표값보다 작은 경우
                else if (currentSum < target) {
                    start++;
                }
                // Case 4: 합이 목표값보다 큰 경우
                else {
                    end--;
                }
            }
        }

        System.out.println(count);
        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');
const input :string[] = fs.readFileSync('/dev/stdin').toString().trim().split( separator: '\n');

const n :number = parseInt(input[0], radix: 10);
const numbers :number[] = input[1].split( separator: ' ').map(Number);

// 숫자 오름차순으로 정렬
numbers.sort( compareFn: (a :number , b :number ) => a - b);

let count :number = 0;

for (let k :number = 0; k < n; k++) {
    const target :number = numbers[k];
    let start :number = 0;
    let end :number = n - 1;

    while (start < end) {
        // 포인터가 자기 자신을 가리키는 경우 스킵
        if (start === k) {
            start++;
            continue;
        }
        if (end === k) {
            end--;
            continue;
        }

        const currentSum :number = numbers[start] + numbers[end];

        if (currentSum === target) {
            count++;
            break;
        } else if (currentSum < target) {
            start++;
        } else {
            end--;
        }
    }
}

console.log(count);
```

Python

스택으로 오름차순 수열 만들기

**문제[011] 스택으로 오름차순 수열 만들기**

시간 제한 2초 | 난이도 실버 Ⅲ | 백준 온라인 저지 1874번

임의의 수열을 스택에 넣었다가 출력하는 방식으로 오름차순 수열을 출력할 수 있는지 확인하고, 출력할 수 있다면 push와 pop 연산을 어떤 순서로 수행해야 하는지를 알아내는 프로그램을 작성해 보자.

↓ 입력

1번째 줄에 수열의 개수 n ($1 \leq n \leq 100,000$)이 주어진다. 2번째 줄에서 n 개의 줄에는 수열을 이루는 1 이상 n 이하의 정수가 1개씩 순서대로 주어진다. 이때 같은 정수가 두 번 이상 나오지는 않는다.

↑ 출력

오름차순 수열을 만들기 위한 연산 순서를 출력한다. push 연산은 +, pop 연산은 -로 출력하고, 불가능할 때는 NO를 출력한다.

예제 입력 1

```
8 // 수열의 개수
4
3
6
8
7
5
2
1
```

예제 출력 1

```
+
+
+
+
-
-
+
+
+
-
-
-
-
-
```

예제 입력 2

```
5
1
2
5
3
4
```

예제 출력 2

```
NO
```



스택

- 마지막으로 입력한 데이터가 가장 먼저 나오는
LIFO방식의 자료구조

스택에 데이터를 넣는 것을 '푸시(Push)'
데이터를 꺼내는 것을 '팝(Pop)'

```
n = int(input())
stack = []
result = []
flag = 0
cur = 1
for i in range(n):
    num = int(input())
    while cur <= num:
        stack.append(cur)
        result.append("+")
        cur += 1

    if stack[-1] == num:
        stack.pop()
        result.append("-")
    else:
        print("NO")
        flag = 1
        break

if flag == 0:
    for i in result:
        print(i)
```

Java, JS

스택으로 오름차순 수열 만들기



Java

```
public class P007 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine());

        Stack<Integer> stack = new Stack<>();
        StringBuilder result = new StringBuilder();

        int currentNum = 1;
        boolean isPossible = true; // 수열 생성 가능 여부를 판단하는 플래그

        for (int i = 0; i < n; i++) {
            int targetNum = Integer.parseInt(br.readLine());

            // 현재 스택에 넣을 숫자가 만들어야 할 숫자보다 작거나 같으면 push
            while (currentNum <= targetNum) {
                stack.push(currentNum);
                result.append("+\n");
                currentNum++;
            }

            // 스택의 top이 만들어야 할 숫자와 같으면 pop
            if (!stack.isEmpty() && stack.peek() == targetNum) {
                stack.pop();
                result.append("-\n");
            }

            // 스택의 top이 만들어야 할 숫자와 다르면 수열 생성 불가
            else {
                isPossible = false;
                break;
            }
        }

        if (isPossible) {
            System.out.println(result.toString());
        } else {
            System.out.println("NO");
        }

        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');
const input : number[] = fs.readFileSync('/dev/stdin').toString().trim().split(' ').map(Number);

const n : number = input[0];
const sequence : number[] = input.slice(1);

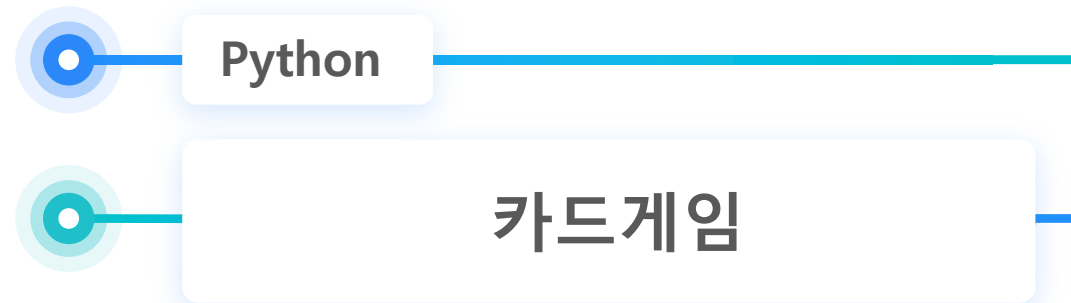
const stack : any[] = [];
const result : any[] = [];
let currentNum : number = 1;
let isPossible : boolean = true;

for (let i : number = 0; i < n; i++) {
    const targetNum : number = sequence[i];

    while (currentNum <= targetNum) {
        stack.push(currentNum);
        result.push('+');
        currentNum++;
    }

    if (stack[stack.length - 1] === targetNum) {
        stack.pop();
        result.push('-');
    } else {
        isPossible = false;
        break;
    }
}

if (isPossible) {
    console.log(result.join('\n'));
} else {
    console.log('NO');
}
```



**문제[013] 카드게임**

시간 제한 2초 | 난이도 실버 IV | 백준 온라인 저지 2164번

N장의 카드가 있다. 각각의 카드는 차례로 1에서 N까지의 번호가 붙어 있으며, 1번 카드가 가장 위, N번 카드가 가장 아래인 상태로 놓여 있다. 이제 다음과 같은 동작을 카드가 1장 남을 때까지 반복한다.

먼저 가장 위에 있는 카드를 바닥에 버린다. 그다음 가장 위에 있는 카드를 가장 아래에 있는 카드 밑으로 옮긴다. 예를 들어 $N = 4$ 일 때를 생각해 보자. 카드는 가장 위에서부터 1, 2, 3, 4의 순서대로 놓여 있다. 1을 버리면 2, 3, 4가 남는다. 여기서 2를 가장 아래로 옮기면 순서가 3, 4, 2가 된다. 3을 버리면 4, 2가 남고, 4를 밑으로 옮기면 순서가 2, 4가 된다. 마지막으로 2를 버리면 카드 4가 남는다. N이 주어졌을 때 가장 마지막에 남는 카드를 구하는 프로그램을 작성하시오.

입력

1번째 줄에 정수 $N(1 \leq N \leq 500,000)$ 이 주어진다.

출력

1번째 줄에 남는 카드의 번호를 출력한다.

예제 입력 1

6 // 카드의 개수

예제 출력 1

4



큐(Queue)

한쪽 끝에서만 삽입이 이루어지고,
다른 한쪽 끝에서는 삭제 연산만
이루어지는 유한 순서 리스트



First in First Out(FIFO) – 선입선출

먼저 들어온 것이 먼저 나가는 형식



일반 queue 사용시 시간 초과

- list는 배열처럼 동작
- 긴 줄에서 맨 앞사람이 빠졌을때 모두 한칸씩 움직이는 것과 같은 동작

```
n = int(input())
queue = list(range(1, n+1))

while len(queue) > 1:
    queue.pop(0)
    queue.append(queue[0])
    queue.pop(0)

print(queue[0])
```

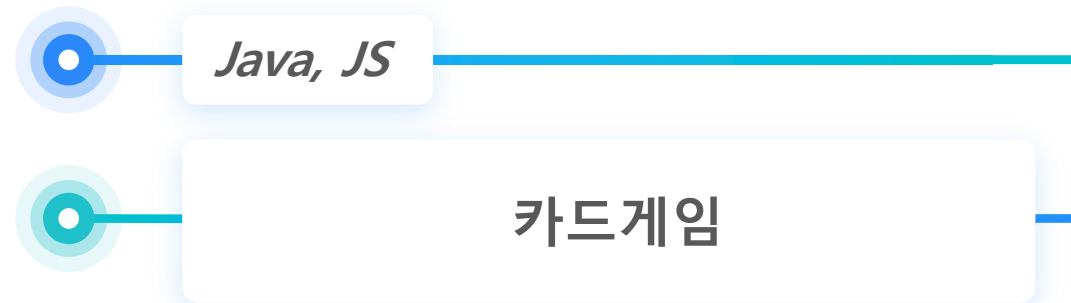
deque 사용으로 시간 초과 해결

- 이중 연결 리스트로 구현
- 기차에서 맨 앞 칸을 떼는 것과 비슷

```
from collections import deque
n = int(input())
queue = deque(range(1, n+1))

while len(queue) > 1:
    queue.popleft()
    queue.append(queue[0])
    queue.popleft()

print(queue[0])
```





Java

```
public class P008 { new *
    public static void main(String[] args) throws IOException { new *
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine());

        // LinkedList를 사용하여 큐를 생성
        Queue<Integer> queue = new LinkedList<>();

        // 1부터 N까지의 카드를 큐에 순서대로 넣음
        for (int i = 1; i <= n; i++) {
            queue.add(i);
        }

        // 카드가 1장 남을 때까지 반복
        while (queue.size() > 1) {
            queue.poll();

            int frontCard = queue.poll();
            queue.add(frontCard);
        }

        // 마지막에 남은 카드 출력
        System.out.println(queue.peek());

        br.close();
    }
}
```

JavaScript

```
const fs = require('fs');
const n : number = parseInt(fs.readFileSync('/dev/stdin').toString().trim(), radix: 10);

const queue : unknown[] = Array.from( arrayLike: { length: n }, mapfn: (_, i : number) => i + 1);
let head : number = 0; // 큐의 시작점을 가리키는 포인터

while (queue.length - head > 1) {
    // 1. 맨 위의 카드를 버림 (실제로 제거하지 않고 head 포인터만 이동)
    head++;

    // 2. 그 다음 맨 위 카드를 맨 아래로 옮김
    const frontCard = queue[head];
    head++;
    queue.push(frontCard);
}

console.log(queue[head]);
```

감사합니다!